# Fase 3 - Modelos de herencia, polimorfismo y gestión de métodos

Presentado por:
Jhon Alejandro Moreno Daza

Programación
Grupo - 213023

Presentado a:
Viky Erika Santanilla Parra

Universidad Nacional Abierta y a Distancia -UNAD
Escuela de Ciencias Básica, Tecnologías e Ingenierías – ECBTI
12 de abril de 2026

Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.

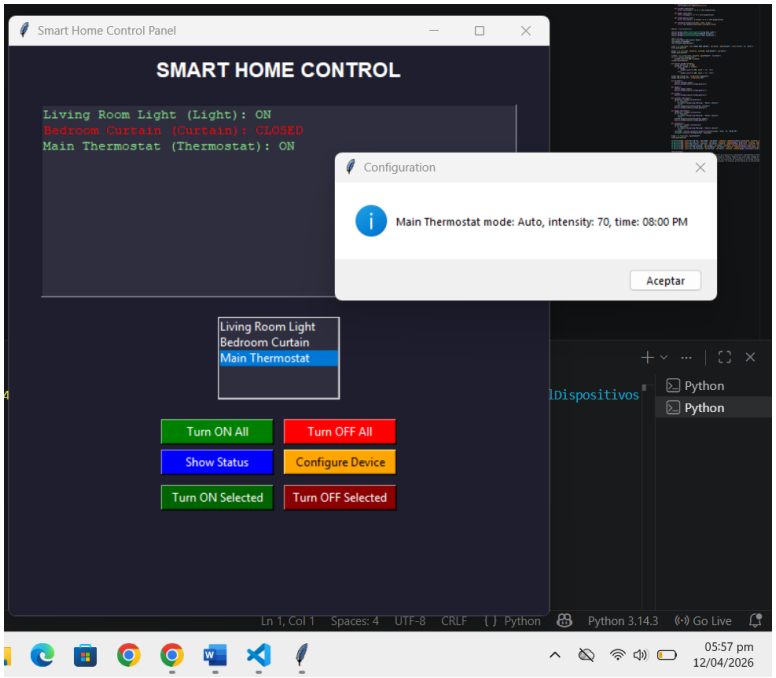
# INTRODUCCIÓN

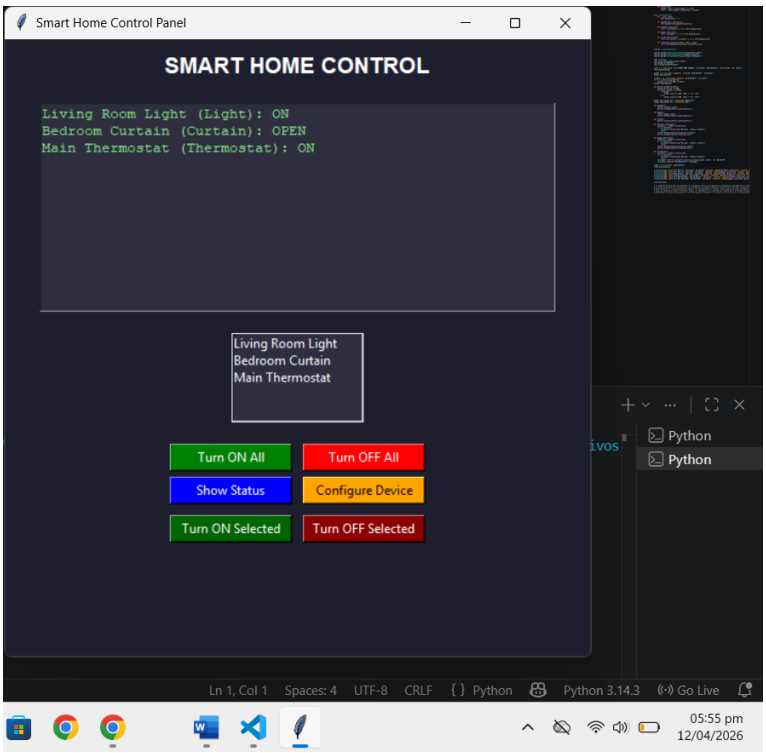
En la actualidad, el desarrollo de software ha evolucionado significativamente gracias a la incorporación de nuevas tecnologías como la Inteligencia Artificial y la Programación Orientada a Objetos. Este trabajo tiene como propósito integrar estos conceptos mediante el diseño e implementación de un sistema de control de dispositivos inteligentes utilizando el lenguaje de programación Python y la biblioteca Tkinter para la creación de interfaces gráficas. A través de este sistema, se aplican principios fundamentales como la herencia, el polimorfismo y la sobrecarga de métodos, permitiendo una estructura flexible, escalable y fácil de mantener.

Además, se complementa el desarrollo práctico con la elaboración de un mapa mental en inglés sobre las principales herramientas de inteligencia artificial utilizadas en el desarrollo de software, con el fin de comprender su impacto en la productividad, la calidad del código y la innovación tecnológica. De esta manera, el trabajo no solo fortalece las habilidades de programación, sino que también promueve el análisis de tendencias actuales en el ámbito tecnológico.

Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.

# Imágenes de prueba del programa




Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.

**Código de programación.**

```
import tkinter as tk
from tkinter import messagebox

class Dispositivo:
    def __init__(self, nombre):
        self.nombre = nombre
        self.estado = False

    def encender(self):
        self.estado = True
        return f"{self.nombre} is ON"

    def apagar(self):
        self.estado = False
        return f"{self.nombre} is OFF"

    def status(self):
        estado = "ON" if self.estado else "OFF"
        return f"{self.nombre}: {estado}"

    def configurar(self, *args):
        if len(args) == 1:
            return f"{self.nombre} configured in {args[0]} mode"
        elif len(args) == 2:
            return f"{self.nombre} mode: {args[0]}, intensity: {args[1]}"
        elif len(args) == 3:
            return f"{self.nombre} mode: {args[0]}, intensity: {args[1]},
time: {args[2]}"
        else:
            return f"{self.nombre} default configuration"


class BombillaInteligente(Dispositivo):
    def status(self):
        estado = "ON" if self.estado else "OFF"
        return f"{self.nombre} (Light): {estado}"


class CortinaInteligente(Dispositivo):
```

Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.

```
    def status(self):
        estado = "OPEN" if self.estado else "CLOSED"
        return f"{self.nombre} (Curtain): {estado}"


class TermostatoInteligente(Dispositivo):
    def status(self):
        estado = "ON" if self.estado else "OFF"
        return f"{self.nombre} (Thermostat): {estado}"


class ControlCentral:
    def __init__(self):
        self.dispositivos = []

    def agregar(self, dispositivo):
        self.dispositivos.append(dispositivo)

    def encender_todos(self):
        return [d.encender() for d in self.dispositivos]

    def apagar_todos(self):
        return [d.apagar() for d in self.dispositivos]

    def estado_general(self):
        return [(d.status(), d.estado) for d in self.dispositivos]

    def configurar_dispositivo(self, index, *args):
        return self.dispositivos[index].configurar(*args)


control = ControlCentral()

control.agregar(BombillaInteligente("Living Room Light"))
control.agregar(CortinaInteligente("Bedroom Curtain"))
control.agregar(TermostatoInteligente("Main Thermostat"))

root = tk.Tk()
root.title("Smart Home Control Panel")
root.geometry("550x580")
root.configure(bg="#1e1e2f")
```

Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.

```
title = tk.Label(root, text="SMART HOME CONTROL", fg="white", bg="#1e1e2f",
font=("Arial", 16, "bold"))
title.pack(pady=10)

output = tk.Text(root, height=12, width=60, bg="#2e2e3f", fg="white")
output.pack(pady=10)

listbox = tk.Listbox(root, height=5, bg="#2e2e3f", fg="white")
for d in control.dispositivos:
    listbox.insert(tk.END, d.nombre)
listbox.pack(pady=10)

def mostrar_estados(estados):
    output.delete(1.0, tk.END)
    for texto, estado in estados:
        if estado:
            output.insert(tk.END, texto + "\n", "on")
        else:
            output.insert(tk.END, texto + "\n", "off")

output.tag_config("on", foreground="lightgreen")
output.tag_config("off", foreground="red")

def encender():
    control.encender_todos()
    mostrar_estados(control.estado_general())

def apagar():
    control.apagar_todos()
    mostrar_estados(control.estado_general())

def estado():
    mostrar_estados(control.estado_general())

def encender_individual():
    seleccion = listbox.curselection()
    if not seleccion:
        messagebox.showwarning("Warning", "Select a device")
        return
    control.dispositivos[seleccion[0]].encender()
    mostrar_estados(control.estado_general())
```

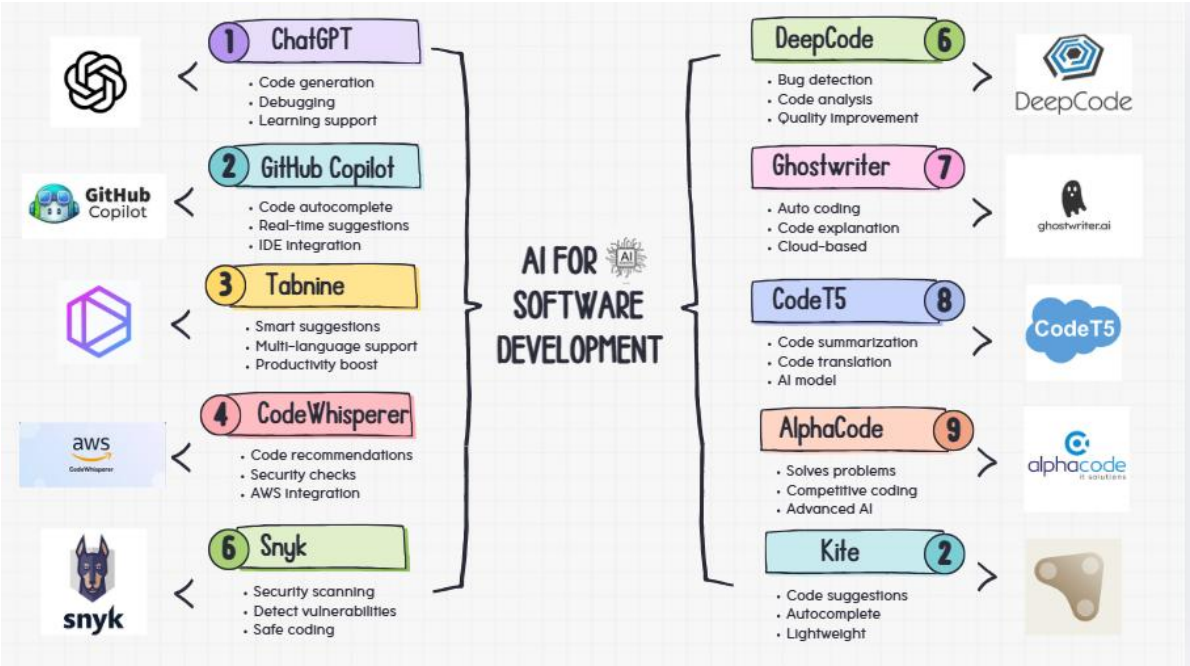
Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.

```
def apagar_individual():
    seleccion = listbox.curselection()
    if not seleccion:
        messagebox.showwarning("Warning", "Select a device")
        return
    control.dispositivos[seleccion[0]].apagar()
    mostrar_estados(control.estado_general())

def configurar():
    seleccion = listbox.curselection()
    if not seleccion:
        messagebox.showwarning("Warning", "Select a device")
        return
    resultado = control.configurar_dispositivo(seleccion[0], "Auto", 70,
"08:00 PM")
    messagebox.showinfo("Configuration", resultado)

frame = tk.Frame(root, bg="#1e1e2f")
frame.pack(pady=10)

tk.Button(frame, text="Turn ON All", bg="green", fg="white", width=15,
command=encender).grid(row=0, column=0, padx=5)
tk.Button(frame, text="Turn OFF All", bg="red", fg="white", width=15,
command=apagar).grid(row=0, column=1, padx=5)
tk.Button(frame, text="Show Status", bg="blue", fg="white", width=15,
command=estado).grid(row=1, column=0, padx=5, pady=5)
tk.Button(frame, text="Configure Device", bg="orange", fg="black", width=15,
command=configurar).grid(row=1, column=1, padx=5, pady=5)
tk.Button(frame, text="Turn ON Selected", bg="darkgreen", fg="white",
width=15, command=encender_individual).grid(row=2, column=0, padx=5, pady=5)
tk.Button(frame, text="Turn OFF Selected", bg="darkred", fg="white",
width=15, command=apagar_individual).grid(row=2, column=1, padx=5, pady=5)

root.mainloop()

# El programa desarrollado corresponde a un sistema de control de
dispositivos inteligentes implementado en Python utilizando la biblioteca
Tkinter para la interfaz gráfica.
```

Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.

```
# Se diseñó aplicando los principios de la programación orientada a objetos,
mediante una clase base llamada Dispositivo, de la cual heredan las clases
BombillaInteligente, CortinaInteligente y TermostatoInteligente,
# permitiendo así la reutilización del código y la escalabilidad del sistema.
Cada dispositivo puede encenderse, apagarse y mostrar su estado, haciendo uso
del polimorfismo para que, aunque se utilicen los mismos métodos,
# cada uno responda de manera diferente. Además, se implementó una simulación
de sobrecarga en el método configurar() mediante el uso de argumentos
variables. La clase ControlCentral se encarga de gestionar todos los
dispositivos,
# permitiendo acciones tanto globales como individuales. Finalmente, la
interfaz gráfica facilita la interacción del usuario mediante botones y
visualización de estados, destacando en colores verde y rojo los dispositivos
encendidos y apagados respectivamente.
```

**Mapa mental.**




Link: https://canva.link/s8gp08fwhc5emf9

Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.

# Referencias bibliográficas

•Van Rossum, G., & Drake Jr, F. L. (2024). El tutorial de Python. Python Software Foundation. https://docs.python.org/es/3.12/tutorial/classes.html. (Apartado: 9.5 a 9.10)

• Rodríguez Guerrero, R., Vanegas, C. A., & Castang Montiel, G. (2020). Python a su alcance (Creación - constructores) . Editorial Universidad Distrital Francisco José de Caldas. Editorial UD. https://research ebscocom.bibliotecavirtual.unad.edu.co/linkprocessor/plink?id=efd0e39 9-de3d-3da8-9598-d7d1e3936c08 (Temas: Herencia y encapsulación)

• Hinojosa Gutiérrez, Á. (2015). Python paso a paso. RA-MA Editorial. https://elibronet.bibliotecavirtual.unad.edu.co/es/ereader/unad/107213?page= 123. (p.127 - 134)

• Lizcano, F. (2024). Herencia en la Programación Orientada a Objetos. [Objeto_virtual_de_aprendizaje_OVA]. Repositorio Institucional UNAD. https://repository.unad.edu.co/handle/10596/60494

• Arango, J. P. (2025). Herencia y polimorfismo. [Objeto_virtual_de_Informacion_OVI]. Repositorio Institucional UNAD. https://repository.unad.edu.co/handle/10596/75877

• Silva, F. D. (2024). Inheritance and Polymorphism. [Objeto_virtual_de_Informacion_OVI]. Repositorio Institucional UNAD. https://repository.unad.edu.co/handle/10596/64522

Prohibida su reproducción y copia sin autorización de la UNAD. UNAD © 2020 derechos reservados.